Dataset Details

AI-Powered Telecom Network Operations Assistant. Every figure below was measured on the full dataset with the project's own code: `scripts/eda_report.py` (evidence in `docs/assets/eda/`, regenerated 2026-10-09), `src/rag/preprocessing.py` and the build manifest `artifacts/manifest.json`.

1. Identity

Item	Value
Name	5G Network Operations (NetOps) Dataset
Source	Hugging Face, `fenar/5g_netops`: https://huggingface.co/datasets/fenar/5g_netops
File	`5G_netops_data_100K.csv.xz` (xz-compressed CSV, UTF-8, header `Temperature (°C)`)
License	Apache-2.0 (as declared on the dataset page)
Size	100,000 rows × 27 columns (6.3 MB compressed); 0 nulls; 0 duplicate rows; 0 values outside plausible KPI ranges
Version used	SHA-256 `006b685455531c414fcb4872aa8e10ca205091c41a0cf7395f276b6d86c96e98`
Committed sample	`data/sample_data/netops_sample_10k.csv`, 9,996 rows (§5)
Alternative considered	RANalyzer (listed in the project plan as an alternative): not used. *(Reason to be added by the author.)*

What the dataset does not contain: no free text (no incident description, root cause or action), no timestamp, no cell or site ID and no region field beyond state, city and ZIP. Those gaps shaped the design (§6).

2. Purpose in the project

Use	How
Retrieval corpus	Each row becomes a "historical case" with a generated narrative (§4.5), embedded for semantic search and indexed for BM25
ML training	Fault-risk classifier and throughput regressor trained on leading-indicator columns only (§6.3)
Baselines	Per-KPI percentile tables (p5–p95, 1,001 points) that define "degraded" and "critical" for both historical and new incidents
Segment statistics	Co-occurrence lifts per category, season and weather, used as contributing-factor evidence

3. Fields

Statistics computed on all 100,000 rows. **Role:** *model feature* = used by the ML models; *co-outcome* = computed by the synthetic generator from other columns, so excluded from features (§6.3); *context* = environment; *identifier* = display and filtering only.

Raw column	Normalized name	Unit	Type	Range	p25 / p50 / p75	Role
Season	`season`		text	4 values	Fall, Spring, Winter, Summer	model feature (context)

Raw column	Normalized name	Unit	Type	Range	p25 / p50 / p75	Role
Cell Availability (%)	`cell_availability_pct`	%	float	74.342 to 99	83.19 / 89.4 / 95.71	model feature
MTTR (hours)	`mttr_hours`	hours	float	0.5 to 10	2.89 / 5.25 / 7.62	model feature
Throughput (Mbps)	`throughput_mbps`	Mbps	float	114.992 to 1283.86	394.3 / 549.21 / 744.45	ML target (performance); co-outcome
Latency (ms)	`latency_ms`	ms	float	4.618 to 175.844	49.31 / 77.22 / 105.39	model feature
Packet Loss Rate (%)	`packet_loss_pct`	%	float	0.1 to 17	3.19 / 6.24 / 9.33	model feature
Call Drop Rate (%)	`call_drop_rate_pct`	%	float	0 to 6.90032	0 / 0 / 1.66	co-outcome (excluded)
Handover Success Rate (%)	`handover_success_pct`	%	float	90 to 100	92.51 / 95.02 / 97.52	model feature
Alarm Count	`alarm_count`	alarms	integer	0 to 26	8 / 10 / 12	model feature
Critical Alarm Count	`critical_alarm_count`	alarms	float	0 to 13.8642	4.57 / 6.75 / 8.42	co-outcome (excluded)
Parameter Changes	`parameter_changes`	changes	integer	0 to 19	4 / 10 / 15	model feature
Successful Configuration Changes (%)	`config_change_success_pct`	%	float	90 to 99	92.24 / 94.49 / 96.75	model feature
Data Usage (GB)	`data_usage_gb`	GB	float	10 to 50	12.72 / 25.16 / 37.59	model feature
User Count	`user_count`	users	integer	100 to 9999	2596 / 5060 / 7526	model feature
Signal Strength (dBm)	`signal_strength_dbm`	dBm	float	-120 to -60	-105.06 / -90.06 / -75	model feature
Jitter (ms)	`jitter_ms`	ms	float	0.1 to 50	12.51 / 25 / 37.5	model feature
Connection Setup Success Rate (%)	`cssr_pct`	%	float	90 to 99	92.49 / 94.96 / 97.47	model feature
Security Incidents	`security_incidents`	incidents	integer	1 to 19	5 / 10 / 15	model feature
Authentication Failures	`auth_failures`	failures	integer	0 to 63	4 / 8 / 18	co-outcome (excluded)
Temperature (°C)	`temperature_c`	°C	float	-10 to 40	10.01 / 17.47 / 23.63	model feature (context)
Humidity (%)	`humidity_pct`	%	float	20 to 90	43.36 / 54.9 / 66.73	model feature (context)

Raw column	Normalized name	Unit	Type	Range	p25 / p50 / p75	Role
Weather	`weather`		text	4 values	Clouds, Rain, Clear, Snow	model feature (context)
Issue Reported	`issue_reported`		yes/no	2 values	no 50,111 · yes 49,889	noise: display only
City	`city`		text	38,066 values	—	identifier / filter
State	`state`		text	59 values	—	identifier / filter
Zip	`zip_code`		integer	503 to 99950	—	identifier (display)
Fault Occurrence Rate (%)	`fault_occurrence_rate_pct`	%	float	0 to 81.721	28.34 / 39.46 / 48.15	ML target (fault risk); co-outcome

4. Preprocessing performed

`python -m scripts.bootstrap` runs every step in order; no LLM is used.

1. **Load and validate** (`src/rag/data_loader.py`): assert the 27 expected columns, read UTF-8, check plausible ranges per KPI (`KPI_SPECS`; 0 violations on the full data).
2. **Normalize** (`src/rag/preprocessing.normalize`): snake_case names, numeric coercion, `issue_reported` to boolean, a case-folded city copy for filtering, and stable IDs `CASE-000001` … `CASE-100000`.
3. **Baselines and bands** (`src/services/kpi_analyzer.py`): severity is **relative to this network**. A KPI is *degraded* when worse than its p75 (p25 for lower-is-worse KPIs) and *critical* when worse than p90 (p10). Percentiles use a mid-rank on a 1,001-point quantile table, so ties are not overstated (50.2% of call-drop rates are exactly 0).
4. **Symptoms and categories.** Each of the 19 severity KPIs maps to one symptom (e.g. `latency_ms` → `high_latency`). Each of the 8 incident categories has a symptom signature. Category score = sum of the severities of its signature symptoms ÷ √(signature size); the primary category is the top score (ties broken by the most extreme KPI) and up to two secondary categories follow. A row with no abnormal KPI is `NORMAL`.
5. **ML training** (`src/services/ml/train.py`): fault-risk classifier and throughput regressor; out-of-fold fault-risk probabilities are stored for every historical case, so a case's risk is never predicted by a model that saw it.
6. **Priority** (`src/services/prioritization.py`): score = 0.4 × severity + 0.3 × impact + 0.3 × risk, with overrides that force at least P2 (critical availability; critical alarms with fault rate; security incidents with authentication failures). Level cut-offs are corpus quantiles: P1 top 5%, P2 next 20%, P3 next 40%, P4 the rest.
7. **Narratives** (`src/rag/documents.py`, template v1). Each case becomes one short document that names only its abnormal KPIs, with severity word, value, percentile and synonyms, so keyword search gets meaningful terms and the embedding focuses on the symptom combination. A real example:

> CASE-000001, MN, Winter, Clouds, -6.8 C. Critical: high data usage 47.5 GB (worse than 95% of records; traffic surge, heavy data); frequent parameter changes 18 changes (worse than 92% of records; frequent config changes, recent changes). Degraded: weak signal -110.1 dBm (worse than 84% of records; poor signal, low rsrp); high alarm count 13 alarms (worse than 83% of records; alarm flood, rising alarm counts);

> failed configuration changes 91.8% (worse than 79% of records; failed changes, rollbacks). Primary issue: Configuration changes; secondary: Congestion and capacity, Radio coverage and signal quality. Load 2,442 users, 47.5 GB. Fault occurrence rate 33.6%. Fault risk Low. Priority P3.

8. **Indexing** (`src/rag/ingest.py`): 384-dimensional `BAAI/bge-small-en-v1.5` embeddings in ChromaDB plus a bm25s keyword index, with metadata (state, city, season, weather, categories, priority, risk band) for filtering. Full-dataset build on a laptop CPU: embeddings 5,698 s, Chroma 278 s, BM25 14 s (`artifacts/manifest.json`).

Resulting label distribution (100,000 cases)

Primary category	Share		Priority	Cases
FAULT_ALARM	15.0%		P1 Critical	5,000
CONFIG_CHANGE	13.7%		P2 High	19,996
AVAILABILITY_OUTAGE	13.2%		P3 Medium	39,995
CONGESTION_CAPACITY	12.8%		P4 Low	35,009
RADIO_COVERAGE	12.0%			
TRANSPORT_QOS	11.8%			
SECURITY_AUTH	11.0%			
MOBILITY_HANDOVER	9.5%			
NORMAL	1.1%			

On average a case has 4.83 abnormal KPIs: in this data nearly every row is an "incident", so ranking by severity matters more than detecting one.

5. Sampling method (10k sample)

`python -m scripts.make_sample` draws about 10% of each primary-category × season stratum with seed 42, kept in the original raw CSV format. Per-stratum rounding gives 9,996 rows; category shares match the full data within 0.01 percentage points. The sample lets the project run without the full download: `python -m scripts.bootstrap --source sample` was verified from a clean folder on 2026-10-09 (18 minutes on a laptop CPU, then the app starts ready with 9,996 cases).

6. EDA findings and limitations

6.1 Evidence that the data is synthetic

Finding	Measured
City names are Faker-style and near-unique	38,066 unique cities; 20,832 appear in only one row; 93,986 unique city + state pairs
States are uniform	59 states/territories, about 1,700 rows each
Most KPI distributions are uniform	e.g. data usage 10–50 GB, signal −120 to −60 dBm, jitter 0.1–50 ms, with evenly spaced quartiles (§3)
Weather is consistent with season	Clear 31,185 · Clouds 31,392 · Rain 31,204 · Snow 6,219; snow never occurs outside winter (0 rows)

Consequence: the city filter usually matches 0–1 rows, so state, season and weather are the useful filters. KPI magnitudes are unrealistic by real-world standards (cell availability 74–99%, median packet loss 6.2%), which is why severity is percentile-relative.

6.2 `Issue Reported` is noise

A gradient-boosting classifier predicts it from all other columns with ROC-AUC **0.501**, and the yes/no split is 49.9% / 50.1%. It is never used as a label or a feature.

6.3 Leakage audit: co-outcome columns

Some columns are computed by the generator from others. The audit predicts each column from the independently generated ("root") columns only (HistGradientBoosting, 3-fold CV, `docs/assets/eda/leakage_audit.csv`):

Column	R^2 from all other columns	R^2 from root columns only	Decision
`fault_occurrence_rate_pct`	0.9996	0.510	excluded; fault-risk target
`auth_failures`	0.990	0.426	excluded
`critical_alarm_count`	0.998	0.390	excluded
`throughput_mbps`	0.997	0.323	excluded; performance target
`call_drop_rate_pct`	0.942	0.193	excluded
every retained KPI feature	—	≤ 0.125	kept
`temperature_c`, `humidity_pct`	0.816, 0.492	0.816, 0.493	kept as context (derived from season by design)

Rule: a column is excluded from model features if it reaches $R^2 \geq 0.15$ from the root columns, is the target, is label noise or is an identifier. `security_incidents` (0.96) and `alarm_count` (0.95) look predictable from *all* columns only because the fault-rate formula contains them; from the roots they score ≈ 0, so they are kept.

6.4 Other limitations

- **No time axis or topology.** Trends ("increasing alarms", "declining handover") cannot be computed; answers say so. State is the only region.
- **Mostly independent KPIs.** Handover success, signal strength, jitter and cell availability have $R^2 \approx 0$ from all other columns, so root causes cannot be learned from correlations. The analysis is evidence-based hypothesis generation: KPI anomalies, segment statistics and runbook knowledge, each labelled with its evidence type.
- **Mass points.** Call drop rate is 0 in 50.2% of rows; cell availability and CSSR are capped at 99.0.

7. Supplementary source: the knowledge base

The dataset has no troubleshooting text, so the project adds **nine runbooks** (`data/knowledge_base/`), one per incident category plus a general NOC triage runbook.

- **Authorship:** drafted with LLM assistance for this project from general, vendor-neutral NOC practice and public standards (3GPP TS 32.450 / TS 28.554 KPI definitions, ITU-T Y.1541), then reviewed by the project author. They are reference runbooks for a capstone, not an operator's validated procedures; disruptive actions are marked *(requires change approval)*.

- **Structure:** YAML front matter (category, title, version, references) and the same sections in each runbook: overview, KPI signature, likely root causes, diagnostic checks, corrective actions, verification, escalation criteria, interactions with other categories, environmental considerations.
- **Chunking:** one chunk per runbook section, 81 chunks in all, embedded in a separate Chroma collection. Agents retrieve sections by role (the analyst: KPI signature and root causes; the recommender: diagnostic checks, corrective actions and escalation) and cite them by ID, e.g. `KB-TRANSPORT_QOS-corrective-actions`.